Backups Sidebar

Protecting Your Project - Every Engine, Plus a Git Mini-Tutorial

VIDEO GAME CREATION

Introduced alongside Lesson 12, but useful starting day one and worth revisiting any time: this Sidebar is a complete, standalone reference for keeping your project safe. It isn't required reading on any one lesson's schedule - hand it out, post it, or point a student to it whenever it's relevant.

Why This Matters

Game engines and IDEs are constantly writing and maintaining project files behind the scenes - not just when you explicitly hit Save. If your computer sleeps, hibernates, or loses power while one of those files is in the middle of being written, that file can come out corrupted. For a game project, a single corrupted file is often enough to make the whole project fail to load. This is not rare or exotic - it's one of the most common ways students lose real work, and it's completely preventable.

The Four-Step Habit

1. Keep a working battery backup. On a desktop, this means a UPS (uninterruptible power supply) - even a small one is enough, as long as it gives you time to shut down cleanly after a power loss. On a laptop, this means an actual working laptop battery, not a laptop that's only ever run with the battery removed or dead and the charger as its sole power source.
2. Save often. Use File > Save, Ctrl-S, or whatever your engine's explicit save command is - don't wait until a session feels "done" to save for the first time.
3. Always cleanly exit the application before powering off, sleeping, or walking away. Fully close the engine/IDE first - never just close the laptop lid or cut the power while it's still open.
4. Once the application is completely shut down, back up your project. This is the step most people skip. A backup only protects you if it happens after every real session, not occasionally.

Backup Methods, In General

Three methods cover almost every situation, and they aren't mutually exclusive - using more than one is a good idea, not overkill:

- **An engine's own export/import:** Some engines offer a built-in way to package your whole project as one file. When it exists, it's convenient - but check what it actually captures (some export tools are for building a playable game, not for backing up your editable project - this is called out per engine below).
- **A plain filesystem copy:** Copy the project folder (with the engine fully closed first) to another drive, a cloud-synced folder, a USB drive, or anywhere else safe. Works for every engine, always captures everything, and is the most universal method - but for a large engine project, it can take real time (also called out per engine below).
- **Version control (git):** A tool built specifically to track every saved change to a project over time, locally and/or backed up to a free hosted service like GitHub or GitLab. More setup than the other two methods, but it gives you something neither one does on its own: the ability to go back to any earlier saved version, not just your most recent backup. See the mini-tutorial later in this Sidebar.

> ⚠ Whichever method(s) you use, always fully close your engine/IDE before copying, exporting, or backing up its project folder. Backing up files that are still open and being actively written to risks copying a corrupted or half-written file - the exact problem backups are meant to prevent.

Engine by Engine

Export/import notes and filesystem-copy notes for each engine used in this course.

Scratch

- **Export / Import:** File > Save to your computer downloads your whole project as a single .sb3 file - this IS Scratch's export format, and it's also how you back up. File > Load from your computer brings an .sb3 file back in. If you're signed in to a scratch.mit.edu account, File > Save also offers to save to your online account, which is a second, automatic off-computer backup worth using alongside the .sb3 file.
- **Filesystem Backup:** An .sb3 file is a single file, not a folder - just copy it wherever you keep backups. Fast, every time, regardless of project size.

Godot Engine

- **Export / Import:** Project > Export in Godot builds a PLAYABLE game package (an .exe, a web build, and so on) - it is not a project backup and can't be re-opened for editing. For an actual backup, use the filesystem copy method below instead.
- **Filesystem Backup:** Copy the entire project folder (the one containing project.godot). Godot projects are plain text/resource files with no large regenerated cache folders to worry about, so this is normally fast even for a multi-lesson project. To reopen a copy, use the Godot Project Manager's Import button and point it at the copied folder's project.godot file.

Unity

- **Export / Import:** Assets > Export Package lets you export some or all of your project's assets as a single .unitypackage file, which Assets > Import Package brings back in. This is a real, supported way to move or archive assets, but it's asset-level, not a true whole-project snapshot (it won't capture Project Settings the same way a full folder copy does) - treat it as a secondary option, not your main backup method.
- **Filesystem Backup:** Copy the project folder, but only the Assets, Packages, and ProjectSettings subfolders (plus the .gitignore if present) actually need to be backed up - Library, Temp, obj, and Logs are all caches Unity regenerates automatically and safely skipped entirely. Including them anyway isn't wrong, just slow and wasteful - Library in particular can be gigabytes of pure cache.

> ⚠ Backing up a Unity project (even trimmed to just Assets/Packages/ProjectSettings) can take a while once a project has a lot of imported art/audio - budget real time for it, and don't start one two minutes before class ends.

GameMaker

- **Export / Import:** File > Export (sometimes shown as "Create Local Package") bundles your entire project into a single portable .yyz archive file. File > Import Project reopens one. This is GameMaker's own purpose-built, single-file backup/transfer format - genuinely convenient, since there's exactly one file to keep track of.
- **Filesystem Backup:** A plain copy of the project folder (containing the .yyp file) also works, and is just as valid as the .yyz export - use whichever is more convenient in the moment.

GDevelop

- **Export / Import:** GDevelop's Export feature (the rocket-ship icon) builds a playable game (web, Windows, and so on) - like Godot's Export, this is not a project backup. Use File > Save (or Save As) to make sure your project is written out as its own project file/folder first, then back that up with a filesystem copy.
- **Filesystem Backup:** Copy the project file and its resources folder together - GDevelop keeps a project's event sheets and a folder of imported images/sounds side by side, and both are needed to reopen it correctly.

TIC-80

- **Export / Import:** TIC-80 projects are typically a single cartridge file (a .tic binary, or a .lua/.p8 text file if you're using the "folder" project mode) - saving your cart with the console's save command IS your export. There's no separate import step: just open the file.
- **Filesystem Backup:** Because a cart is one small file, copying it anywhere is close to instant. This is the fastest backup of any engine in this course.

001 Game Creator

- **Export / Import:** 001 Game Creator does not have a documented project export/import format for backup purposes (see this course's own Lesson 10 Sidebar on why undocumented-but-tempting built-ins in this engine are treated with extra caution). Use the filesystem copy method as your primary backup.
- **Filesystem Backup:** Copy the entire project folder. 001 Game Creator projects are Windows-only and generally modest in size for a course project, so a full folder copy is the straightforward, reliable option here.

Unreal Engine

- **Export / Import:** Content Browser > right-click an asset (or a folder of assets) > Asset Actions > Migrate copies selected assets, with their dependencies, into another project. It's genuinely useful for moving specific art/blueprints between projects, but it is not a whole-project backup tool - it only grabs what you select.
- **Filesystem Backup:** Copy the project folder, but only Content, Config, and Source (if present), plus the single .uproject file, actually need to be backed up - Binaries, Intermediate, Saved, and DerivedDataCache are all regenerated automatically by the engine and safe to skip entirely.

> ⚠ Even trimmed down, Unreal projects can still be large once a project has real content, and a full (untrimmed) copy that includes the regenerated cache folders can take a genuinely long time and a lot of disk space for no benefit - always exclude Binaries/Intermediate/Saved/DerivedDataCache.

Git: A Local Backup You Can Rewind

Git is a version control system: a tool that keeps a running history of every saved checkpoint of your project, so you can always go back to how things looked at any earlier checkpoint - not just your single most recent backup. It's free, it runs entirely on your own computer (no account or internet connection required to use the core features below), and it's genuinely one of the most common tools in professional software and game development. This is a complete, start-to-finish walkthrough of the core workflow.

1. Install Git

Download git from git-scm.com (choose the installer for your operating system) and run it, accepting the default options unless you have a specific reason not to. This gives you the git command, usable from a terminal/command prompt (or from your engine's own built-in terminal, if it has one).

2. Create Your Project

This is usually already done: it's just your normal engine project folder, created the normal way (a new Scratch/Godot/Unity/GameMaker/etc. project, saved once so the folder and its files actually exist on disk). Git doesn't need anything special done at project-creation time - it attaches to a project folder that already exists, whenever you're ready for the next step.

3. Initialize the Repository

Open a terminal/command prompt, navigate into your project folder, and run:

```
cd path/to/YourGameProject
git init
```

This turns the folder into a git repository - it creates a hidden .git folder that will hold your project's entire saved history from this point forward. Nothing about your actual project files changes; this step only adds tracking.

4. Stage Your Changes (git add)

Before git will save a checkpoint, you tell it which changed files to include - this is called staging. To stage everything in the project folder:

```
git add .
```

The period means "everything in and under this folder." You can also stage individual files by name (git add scenes/Main.tscn) if you only want to include specific changes, but for a backup habit, staging everything with git add . is the simplest reliable choice.

5. Commit (Save a Checkpoint)

A commit is one saved checkpoint in your project's history, with a short message describing what changed. After staging, commit with:

```
git commit -m "Lesson 6: added win sound and score tracking"
```

Write a short, specific message every time - "stuff" or "update" tells future-you nothing; "Lesson 6: added win sound and score tracking" tells you exactly what this checkpoint represents when you're looking back through history later. Repeat steps 4 and 5 (git add ., then git commit -m "...") each time you reach a point worth saving - the end of a lesson, a working feature, right before trying something risky.

6. View Your History (git log)

To see every checkpoint you've saved, in order, run:

```
git log
```

This lists every commit with its message, author, date, and a unique ID (a long string of letters and numbers, called a hash) - you only ever need the first several characters of a hash to refer to that commit. Add --oneline for a shorter, one-line-per-commit view that's easier to scan:

```
git log --oneline
```

7. Reverting to a Previous Version

Git gives you two genuinely different ways to go back to an earlier commit, depending on what you actually want:

Just look at an old version, temporarily

To check out an old commit and look around (useful for comparing, or grabbing something you deleted) without losing your current work permanently:

```
git checkout <commit-hash>
```

This puts your project folder's files into the state they were in at that commit. To get back to your most recent work afterward:

```
git checkout main
```

Permanently go back to an old version

To actually discard everything after a certain commit and make that commit your project's new current state:

```
git reset --hard <commit-hash>
```

> ⚠ git reset --hard permanently discards any commits after the one you reset to, along with any uncommitted changes currently in your project folder. There is no built-in undo for this. If you're not sure, use git checkout first to look around, or make one more commit of your current state before resetting - that way nothing is ever truly lost, it's just one more checkpoint back in the history.

Beyond the Basics (Advanced, But Common)

Two things worth knowing exist, even though they go beyond this mini-tutorial's scope:

- **Recovering a single file:** You don't have to revert your whole project to get back one file from an old commit - git checkout <commit-hash> -- path/to/file restores just that one file from that commit, leaving everything else in your project exactly as it currently is.
- **Syncing across multiple computers with GitHub or GitLab:** A free account on a hosted service like GitHub or GitLab gives your local git repository a second home in the cloud - you push your commits there, and can pull them down again on a different computer. This is how working on the same project from two different computers, or sharing a project with someone else, is commonly handled. Genuinely more setup than everything covered above (it involves creating an online account and connecting your local repository to it), but it's a completely normal, everyday part of how real software and game teams work - worth learning once the basics above feel comfortable.

Quick Reference

```
git init                          start tracking this folder
git add .                         stage all changes
git commit -m "message"           save a checkpoint
git log  (or: git log --oneline)  view history
git checkout <hash>               look at an old commit (temporary)
git checkout main                 return to your latest work
git reset --hard <hash>           permanently go back (careful!)
git checkout <hash> -- <file>     recover just one file (advanced)
```